

Test e buone pratiche

Test e buone pratiche

Come scrivere PHP piu affidabile, leggibile e facile da mantenere.

Scrivere codice leggibile

Il primo passo per avere codice affidabile e renderlo leggibile.

Scegli nomi chiari:

```
<?php
$prezzoFinale = 18.50;
```

E meglio di:

```
<?php
$pf = 18.50;
```

Funzioni piccole

Una funzione piccola e piu facile da capire e provare.

```
<?php
function calcolaSconto(float $prezzo): float {
    if ($prezzo > 50) {
        return 10;
    }

    return 0;
}
```

Questa funzione fa una cosa: decide lo sconto.

Validare gli input

I dati che arrivano da form, URL o file esterni vanno controllati.

```
<?php
$email = $_POST["email"] ?? "";

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email non valida";
    exit;
}
```

La validazione evita errori e migliora la sicurezza.

Un test automatico semplice

Un test automatico verifica che una parte del codice funzioni come previsto.

```
<?php
function somma(int $a, int $b): int {
    return $a + $b;
}

if (somma(2, 3) !== 5) {
    echo "Test fallito";
    exit(1);
}

echo "Test riuscito";
```

Nei progetti veri si usano strumenti come PHPUnit, ma l'idea è la stessa: controllare il risultato atteso.

Sicurezza essenziale

Abitudini importanti:

- valida i dati in ingresso
- usa `htmlspecialchars` quando stampi dati in HTML
- usa query preparate per il database
- non salvare password in chiaro
- non mostrare errori tecnici in produzione

La sicurezza non è un capitolo separato da ricordare alla fine. È una serie di piccole scelte in ogni pagina.

Rendere un test più leggibile

Anche senza strumenti esterni puoi separare preparazione, azione e controllo.

```
<?php
function calcolaTotale(float $prezzo, int $quantita): float {
    return $prezzo * $quantita;
}

$risultato = calcolaTotale(5, 3);

if ($risultato !== 15.0) {
    echo "Test fallito: totale non corretto";
    exit(1);
}

echo "Test riuscito";
```

Il test dice chiaramente cosa si aspettava.

Buone pratiche quotidiane

Non servono regole complicate per migliorare subito il codice. Parti da queste:

- una variabile deve avere un nome comprensibile
- una funzione deve avere un compito principale
- i dati esterni vanno controllati prima di usarli
- gli errori vanno letti, non ignorati
- il codice duplicato spesso merita una funzione

Prima di considerare finita una pagina

Prova almeno un caso normale, un caso vuoto e un caso sbagliato. Per esempio, se ricevi un'email, prova un'email valida, un campo vuoto e un testo che non è un'email.